

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name : M Kavya

Reg. No : 24011101061

Experiment 2

Hyperparameter Optimization of a Multilayer Perceptron for Image Classification

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image pre-processing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28 itemize

4. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

5. Source Code

GitHub Repository

6. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

7. Results

7.1 Baseline Model

Metric	Value
Accuracy	0.8778
Precision (macro)	0.8777
Recall (macro)	0.8778
F1-score (macro)	0.8769

The baseline model achieved 87.78% test accuracy.

7.2 Best Hyperparameters Found

Search Method	RandomizedSearchCV
Candidates Sampled	8
Cross-validation Folds	3
Search Subset Size	10,000 training samples
Total Fits	24

Hyperparameter	Best Value
Hidden Layers	3
Neurons	32
Activation	tanh
Dropout	0.2
Optimizer	RMSProp
Learning Rate	0.001
Batch Size	32
Epochs	30
Best CV Accuracy	0.8418

7.3 Optimized Model (Retrained on Full Training Set)

Metric	Baseline	Optimized
Accuracy	0.877800	0.862600
Precision (macro)	0.877661	0.864324
Recall (macro)	0.877800	0.862600
F1-score (macro)	0.876873	0.863070

8. Plots with Interpretation

8.1 Sample Images



Figure 1: Sample Fashion-MNIST training images with class labels.

The image shows 10 representative sample images from the 10 class labels. Although as complete images they are distinguishable, due to pixel-level similarities, some confusions may occur even in good models.

8.2 Class Distribution

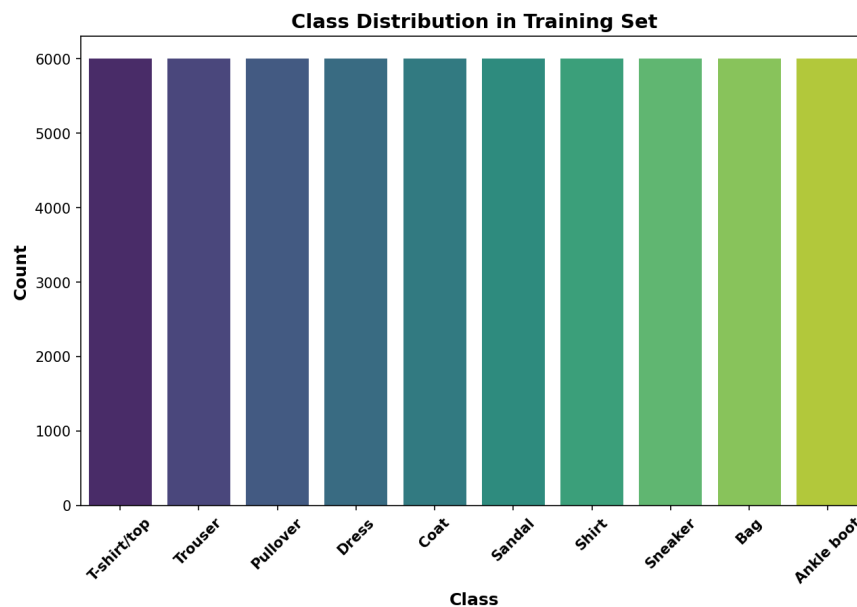


Figure 2: Class distribution in the training set.

Shows that the classes are perfectly balanced. So there is no scope for bias towards any particular class.

8.3 Baseline Training History

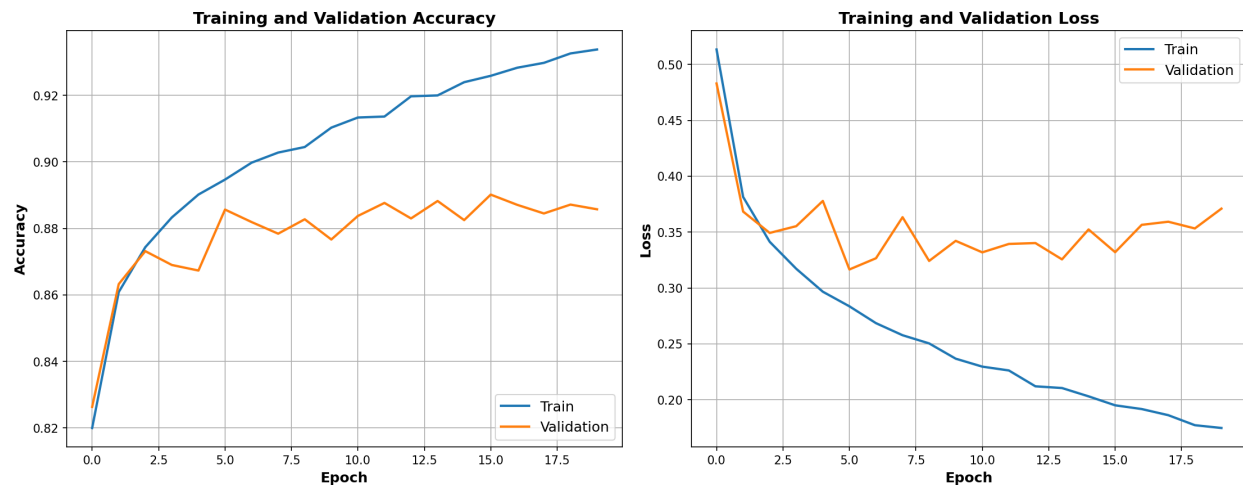


Figure 3: Baseline model training and validation accuracy/loss over 20 epochs.

There is a visible growing gap between training and validation accuracy as well as loss after roughly 10 epochs, indicating mild overfitting. The oscillations in validation loss suggest that the model might be sensitive to noise.

8.4 Baseline Confusion Matrix

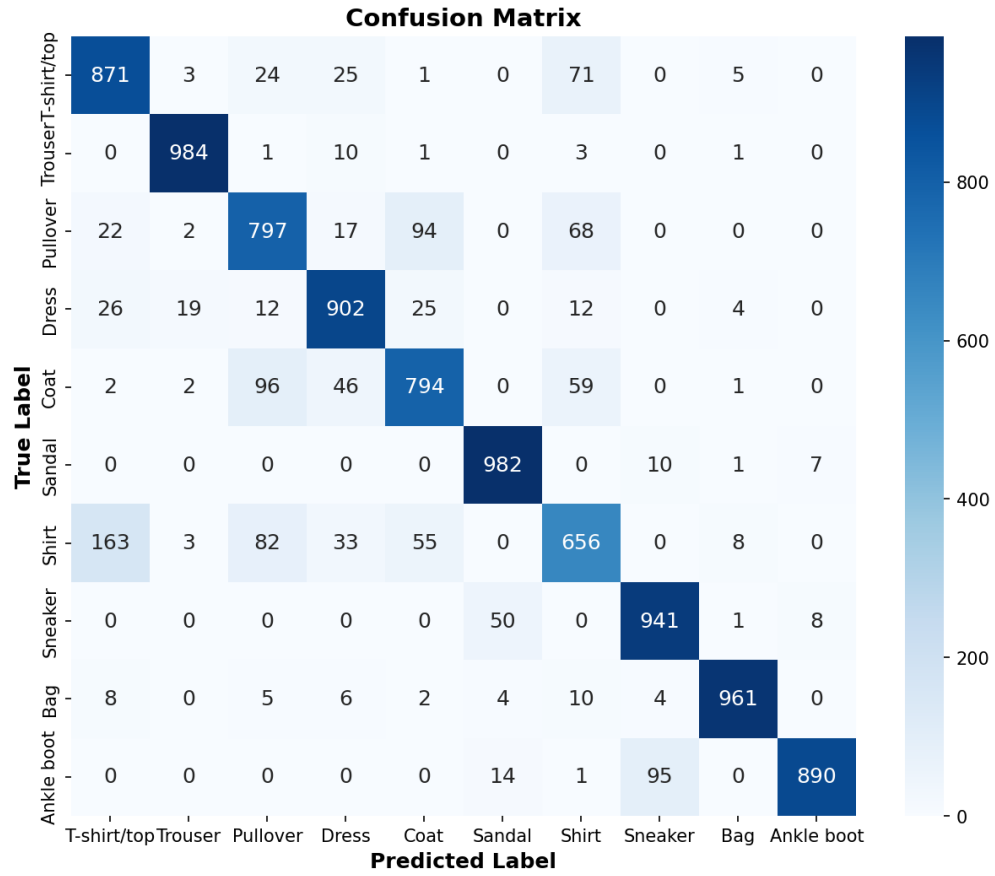


Figure 4: Confusion matrix for the baseline model on the test set.

The matrix shows that most misclassifications are between t-shirts, shirts, pullovers and coats which have similar shapes. Distinct shapes like sandals, trousers and bags are more accurately classified.

8.5 Optimized Training History

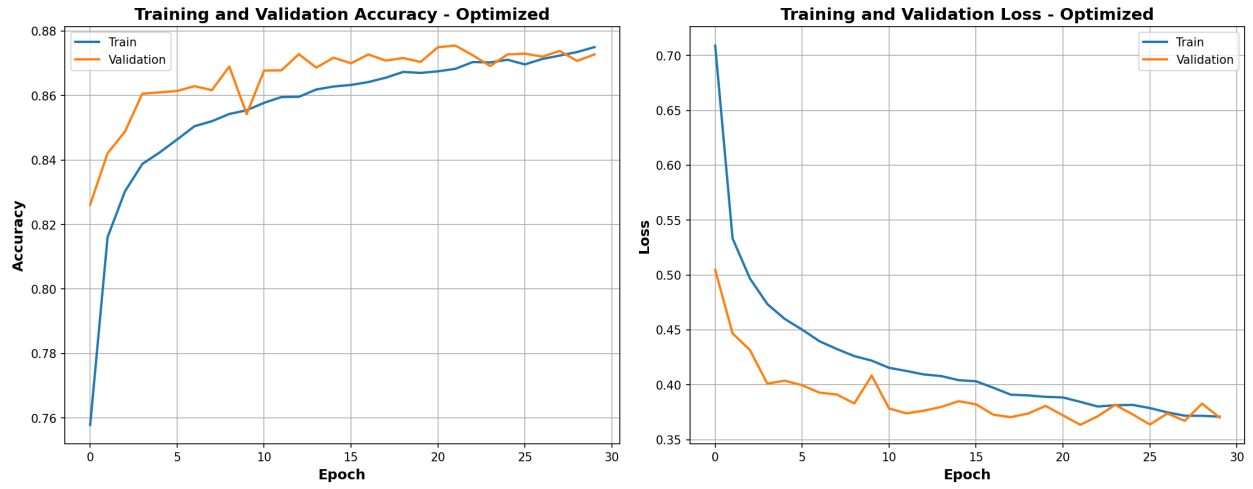


Figure 5: Optimized model training and validation accuracy/loss over 30 epochs.

The gap between training and validation curves has reduced significantly, indicating better generalization. The model was also trained for more epochs without any degradation.

8.6 Optimized Confusion Matrix

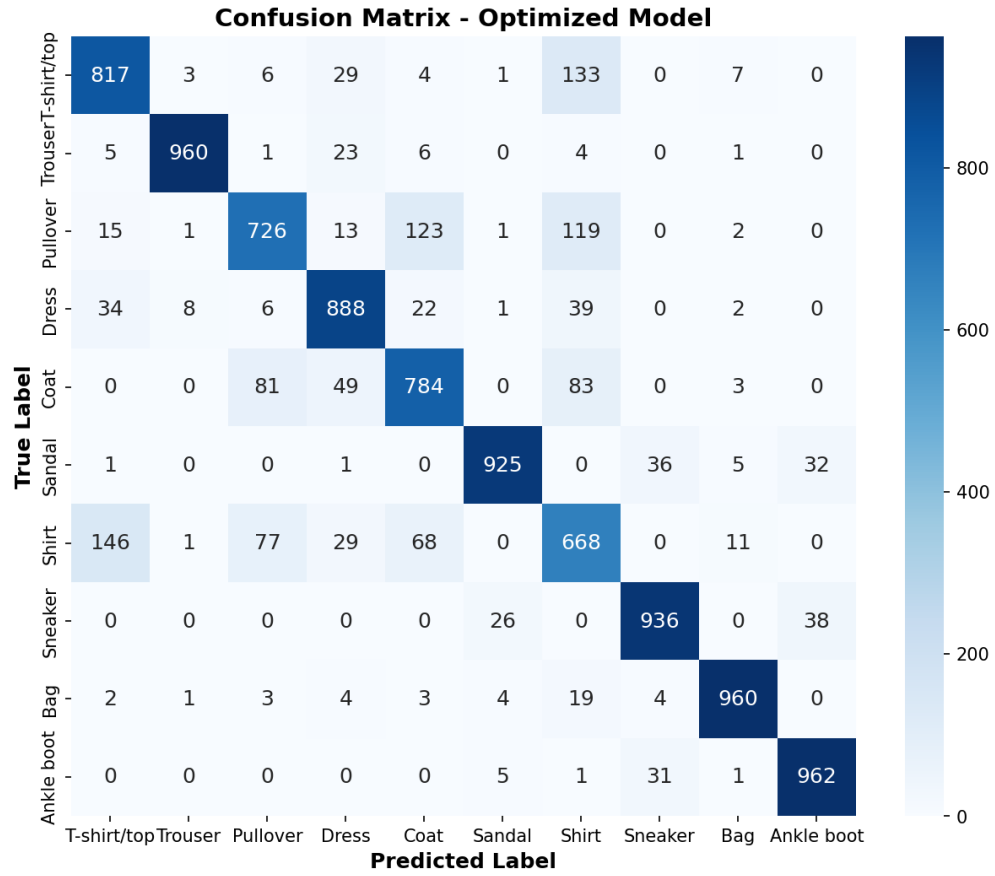


Figure 6: Confusion matrix for the optimized model on the test set.

The classification of the similar shaped tops has improved slightly but it still remains difficult, especially for comparisons with the shirt class. The classifications for the other classes have improved.

8.7 Performance Comparison

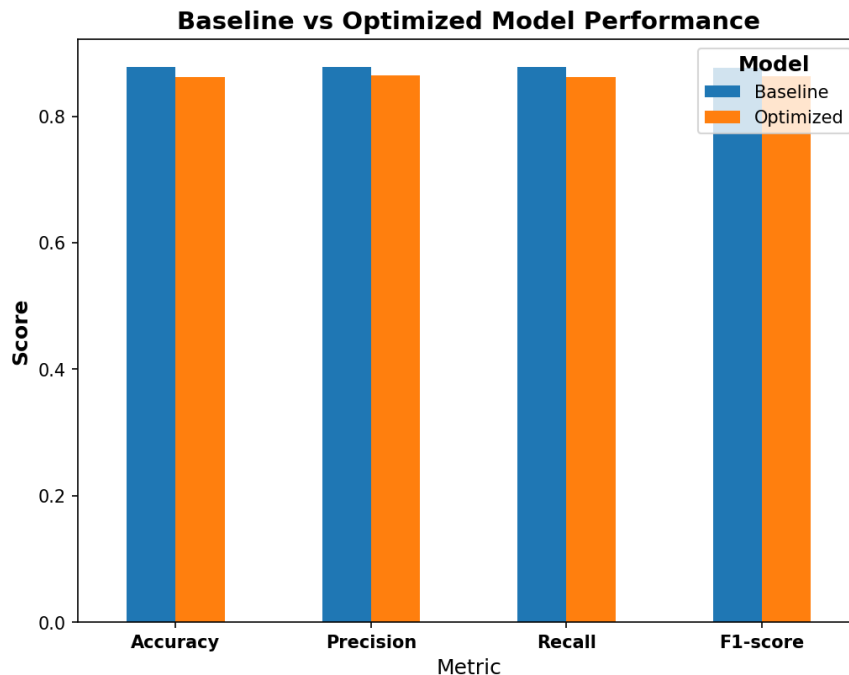


Figure 7: Baseline vs. optimized model performance across all metrics.

Although the performance metrics are slightly higher for the baseline model, the optimized model still shows better generalization by significantly reducing overfitting. Therefore, the optimized model is preferred.

9. Discussion

1. Which optimization method was used?

RandomizedSearchCV with SciKeras wrapper was used. 8 random combinations were evaluated using 3-fold cross-validation on a subset of 10,000 training samples.

2. Which hyperparameters were selected?

The best hyperparameters found were: 3 hidden layers with 32 neurons each, `tanh` activation, dropout rate of 0.2, RMSProp optimizer with learning rate 0.001, batch size 32, and 30 epochs.

3. Did optimization improve performance?

The optimized model achieved a test accuracy of 86.26% compared to the baseline 87.78%. Although there is a slight decrease in performance, the optimized model shows significantly better generalization which is preferred.

4. Which hyperparameter had the greatest impact?

The combination of **activation function (tanh)** and **smaller network size (32 neurons, 3 layers)** with dropout had the greatest impact. This combination made the model more regularized but performance decreased a little compared to baseline.

5. Compare Grid Search and Randomized Search.

RandomizedSearchCV is faster and more efficient than GridSearchCV as it samples a fixed number of combinations from the search space instead of trying all possible combinations. It is particularly useful when the hyperparameter search space is large.

6. Recommend the best MLP configuration.

For higher accuracy, the baseline model (128-64 neurons with ReLU and Adam) is better. However, the optimized model with dropout and tanh offers better generalization which is recommended.

7. Code the perceptron learning algorithm for the XOR gate using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python's built-in packages. Analyze and identify the pattern that prevents the algorithm from converging.

<https://github.com/Kavya-1709/Deep-Learning---Laboratory/tree/main/DL%20Lab%2020-%20Multi%20Layer%20Perceptron/XOR%20Gate> XOR is non-linear problem which prevents a single perceptron from being able to solve it. This is because, there exists no single line which can classify all four points correctly as points belonging to same class lie diagonal to each other.

10. Conclusion

In this experiment, a Multilayer Perceptron was successfully implemented on the Fashion-MNIST dataset. The baseline model achieved a higher accuracy of 87.78%. Hyperparameter optimization using RandomizedSearchCV helped find a more regularized configuration, resulting in better generalization despite a slight decrease in test accuracy to 86.26%. Hence, a trade-off between performance and generalization has been observed.

11. References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. H. Xiao, K. Rasul and R. Vollgraf, "Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms," arXiv:1708.07747, 2017.
4. TensorFlow / Keras Documentation: https://www.tensorflow.org/api_docs
5. SciKeras Documentation: <https://www.adriangb.com/scikeras/>
6. Scikit-learn Documentation: `RandomizedSearchCV`.